STUDENT NAME: AMANDEEP	MODULE: My Practice	CODE: TEST_001_AI
BATCH: ABES_JAS26_FSD JAVA_IIIYear_Batch 02	COURSE: Software Testing Debugging Concepts using AI	STATUS: In Progress

Practice Worksheet – Software Testing & Debugging Concepts using AI

Comprehensive practical test engineering suite covering Factorial boundary testing, Email Regex validation, Arithmetic Exception handling, and Parameterized parity test suites.

KEY LEARNING OBJECTIVES:

- Implement Boundary Value Analysis (BVA) and Equivalence Partitioning for numerical algorithms.
- Design robust regex pattern test suites for institutional email domain validation.
- Triage and assert runtime arithmetic exceptions using JUnit 5 assertions.
- Develop data-driven parameterized unit tests covering positive, negative, and extreme integers.

STEP-BY-STEP WORKSHEET INSTRUCTIONS:

Step 1: Analyze the target Java function and identify its input domain and edge boundaries.
Step 2: Formulate test cases covering normal, boundary, invalid, and extreme scenarios.
Step 3: Write standard JUnit 5 test methods utilizing `@Test`, `@ParameterizedTest`, and `assertThrows`.
Step 4: Verify complete test coverage, assertion accuracy, and exception handling.

EXERCISE 3.1: Q001: BOUNDARY VALUE ANALYSIS FOR FACTORIAL CALCULATION

Problem Statement: A Java method `long calculateFactorial(int n)` calculates factorials. Design an exhaustive test suite using Boundary Value Analysis and Equivalence Partitioning covering: valid small values (n=0, 1, 5), valid maximum limit before long overflow (n=20), overflow boundary (n=21), and invalid negative numbers (n=-1, n=-10) throwing IllegalArgumentException.

Reference Baseline / Code / Scenario:

```
@Test void testFactorialZero() { assertEquals(1, MathUtils.calculateFactorial(0)); }
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Boundary & Partition Matrix: _____

[2] JUnit 5 Test Case Implementation: _____

[3] Overflow & Exception Assertions: _____

EXERCISE 3.2: Q002: EMAIL DOMAIN VALIDATION & EQUIVALENCE PARTITIONING

Problem Statement: Write a test specification for `boolean isValidCollegeEmail(String email)` which must only accept student emails matching the format `^[A-Za-z0-9._%+-]+@abes\.ac\.in$`. Define test matrices for: (1) Valid institutional emails, (2) Unauthorized public domains (@gmail.com), (3) Malformed addresses (missing @, double dots), and (4) Injection payloads.

Reference Baseline / Code / Scenario:

```
@ParameterizedTest @ValueSource(strings = {"student@abes.ac.in", "aman.24j@abes.ac.in"}) void testValid(String e) { assertTrue(EmailValidator.isValid(e)); }
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Equivalence Partition Classes: _____

[2] Regex Assertion Test Suite: _____

[3] Malicious / Edge Case Payloads: _____

EXERCISE 3.3: Q003: ARITHMETIC EXCEPTION HANDLING & ZERO-DIVISION TRIAGE

Problem Statement: An invoice calculation engine performs `BigDecimal divide(BigDecimal a, BigDecimal b)`. Construct unit tests to verify that dividing by zero raises `ArithmeticException` with an informative error message. Demonstrate both JUnit 5 `assertThrows()` and proper graceful handling via try-catch fallback to zero in service implementations.

Reference Baseline / Code / Scenario:

```
assertThrows(ArithmeticException.class, () -> calculator.divide(10, 0));
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Exception Assertion Test: _____

[2] Graceful Service Fallback Logic: _____

[3] Precision & Rounding Verification: _____

EXERCISE 3.4: Q004: PARAMETERIZED UNIT TESTING FOR PARITY (ODD OR EVEN)

Problem Statement: Implement a parameterized unit test for `boolean isEven(int number)`. Using JUnit 5 `@ParameterizedTest` with `@CsvSource`, validate: (1) Even positive integers, (2) Odd positive integers, (3) Zero, (4) Negative even integers (-4), (5) Negative odd integers (-7), and (6) `Integer.MAX_VALUE` and `Integer.MIN_VALUE`.

Reference Baseline / Code / Scenario:

```
@ParameterizedTest @CsvSource({"2, true", "3, false", "0, true", "-4, true", "-7, false"}) void testParity(int num, boolean expected) { assertEquals(expected, MathUtils.isEven(num)); }
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Parameterized Test Annotation: _____

[2] Test Data Inputs & Expected Output: _____

[3] Edge-Case Extremes Verification: _____

SELF-REVIEW & VERIFICATION CHECKLIST:

- [ ] **Boundary Coverage:** Tests cover minimum, nominal, maximum, and extreme out-of-bound inputs.
- [ ] **Exception Assertions:** assertThrows explicitly verifies expected exception types and messages.
- [ ] **Parameterized Rigor:** Data-driven tests run across diverse positive, negative, and zero sets.
- [ ] **Mock & Isolation:** Tests are deterministic, isolated, and self-contained.
- [ ] **Assertion Quality:** Assertions verify exact return values and side-effects.

CRITICAL REFLECTION & LEARNING SYNTHESIS:

- *Why is testing boundary edges (e.g. n=20 vs n=21 in Factorial) critical for preventing silent data corruption?*
- *How does using `@ParameterizedTest` enhance test coverage while drastically reducing test code duplication?*

Reflection Notes:

_____

_____

COMPLETION STATUS:	EVALUATOR & MENTOR REMARKS:
[ ] COMPLETED (Meets All Solution Criteria) [ ] NEEDS REVISION (Resubmit Corrected Work) **Evaluator Signature:** _____	Feedback Notes: _____ Rubric Evaluation: [ ] Correctness [ ] Code Quality [ ] Best Practices